What Pretraining Doesn't Learn: Current-Value Retrieval Is Latent in Pretrained LLMs

Anonymous ACL submission

Abstract

Language models pretrained on trillions of tokens cannot reliably retrieve the current value of a variable that has been overwritten multiple times in their own input. Across 20 pretrained models (14 open-weight from 270M to 9B parameters, 6 proprietary frontier), this failure manifests as a general loss of position-indexed retrieval: ceiling accuracy on the first value, collapsing accuracy on the current value, and near-floor accuracy on intermediate positions of the update stream. The failure is not architectural. A 7.4-million-parameter LoRA — 0.24% of one model's weights — restores accuracy to 93–100% on harder held-out cells; a training-free format change produces similar recovery. Across 79 published checkpoints from two model families, the failure is present at the earliest pretraining checkpoint we can evaluate and survives every post-training stage. The capability is present in pretrained models but not surfaced by current pretraining objectives. ⟨**NOTE:** If §7 lands: add a clause noting whether pre- and post-LoRA mechanism analyses localise to the same attention-head clusters — evidence that LoRA amplifies an existing circuit rather than installing new structure.⟩

Stream (K=3, N=3, randomly interleaved):

```
color: red   size: large   shape: round
size: small   color: blue   shape: square
shape: triangle   color: green   size: medium
```

FVQ: What was the *first* value of `color`? → **red**
CVQ: What is the *current* value of `color`? → **green**

Figure 1: Task illustration (K=3, N=3). The stream randomly interleaves updates for all K keys. To answer CVQ correctly, a model must isolate `color` entries from competing keys and identify the most recent one among them.

1 Introduction

Language models are increasingly deployed in settings where the value of a variable changes over time and the model must use the current value, not an earlier one. Multi-turn assistants must respect the most recent user correction; retrieval-augmented systems must use the latest version of a document; tool-augmented agents must read the current contents of a variable after a sequence of writes. Failures of this skill have been documented at the application level — Laban et al. (2025) report substantial degradation in multi-turn dialogues as state updates cascade — but the underlying mechanism, and whether the failure is intrinsic to current models, is not understood. We isolate this skill with a minimal controlled instance of the broader deployment problem.

Our task interleaves K keys each receiving N value updates in a single stream and queries either the first value of a target key (FVQ) or its current value (CVQ); Figure 1 shows a small instance with K=3, N=3. Both queries are trivially solvable in principle: the entire stream is present in context, FVQ asks for the first occurrence of the target key, and CVQ asks for the last. Failure on this task is therefore evidence of a retrieval problem — the model cannot locate the relevant entries in its own input — not a context-length or memory limitation.

We evaluate 20 pretrained models spanning 14 open-weight checkpoints (270M–9B parameters) and 6 proprietary frontier models. At a representative cell per model block, FVQ−CVQ gaps range from +6 to +53 percentage points, with the majority exceeding +20 pp (Table 1, Figure 2). At low stream load, models systematically favor the first value (*primacy regime*). In several smaller open-weight models at high load, the gap inverts — current-query accuracy exceeds first-query accuracy — but both remain well below ceiling (*reversal regime*). Neither regime represents correct retrieval; both are deviations from the answer the prompt asks for. When we ask for any intermediate position k of a target key ($1 < k < N$), baseline accuracy drops below 5 % within two positions of the stream start (Figure 3; full sweep across six cells in Appendix I), indicating a general loss of position-indexed retrieval rather than a first-vs-current confusion.

The failure is not an architectural limit. A LoRA adapter (Hu et al., 2022) — 0.24% of Qwen2.5-3B-Instruct's parameters, trained on 18,000 examples drawn from a small training grid (against an 18-trillion-token pretraining budget; Qwen Team, 2025) — restores both FVQ and CVQ accuracy to 93–100% on substantially harder held-out cells, well beyond the training grid in both K and N (Table 3). The same LoRA, trained only on one value distribution, transfers to a held-out distribution without modification (Table 4), and recovers the intermediate-position retrieval that the base model lacks (Figure 3). An inference-time format intervention — replacing the plain key:value stream with positionally-anchored alternatives — produces similar recovery without any training (Table 2); this is not the stylistic adjustment small-data alignment achieves (Zhou et al., 2023), but a recovery of a specific mechanistic capability. The recovery is task-specific: a control LoRA trained on arithmetic does not close the gap on update tracking ⟨**NOTE:** pending arithmetic control⟩. Two interventions, one minimal-training and one training-free, converge on the same outcome. The capability is present in the architecture; pretraining does not surface it by default.

We make the following contributions:

- **Behavioral characterization.** We document a robust FVQ–CVQ gap across 20 pretrained models, including a previously-undocumented reversal regime at high load and a general failure of position-indexed retrieval (§4).
- **Two independent recoveries.** We show the failure is recoverable via two independent interventions — a task-specific LoRA and an inference-time format change — demonstrating the capability is latent rather than absent (§5).
- **Training-dynamics analysis.** We trace the failure across 79 pretraining and post-training checkpoints from two model families, showing it is present from the earliest available pretraining step and survives every documented training stage (§6).
- **Mechanism.** We identify the attention-head clusters carrying the bias pre-intervention ⟨**NOTE:** post-LoRA mechanism and pre/post comparison pending⟩ (§7).

Together, these results frame a methodology — **characterise**, **recover**, **locate** — for identifying latent capabilities pretraining does not develop.

2 Related Work

Retrieval failures in LLMs. Application-level work documents that LLMs struggle as in-context state changes: multi-turn dialogues degrade as state updates cascade (Laban et al., 2025); entity-state tracking is unevenly present across models (Kim and Schuster, 2023; Kim et al., 2024; Rezaee et al., 2025); and most closely, Wang and Sun (2025) construct a 46-key stream with up to 400 updates and a single current-value query, reporting a log-linear accuracy decline they conclude is fundamental. Static long-context retrieval has parallel failures: a U-shaped accuracy curve (Liu et al., 2024), an attention sink on initial tokens (Xiao et al., 2024), and benchmarks such as RULER (Hsieh et al., 2024) and HELMET (Yen et al., 2024) probing retrieval of static facts under noise. Theoretical results constrain transformer computation (Hahn, 2020; Merrill and Sabharwal, 2023; Fagnou et al., 2024) but apply to compositional state tracking, which our destructive-update setting does not invoke. Format sensitivity can shift accuracy by up to 76 percentage points (Sclar et al., 2024); our format intervention is a controlled instance. We extend this line in three ways: testing both first-value and current-value queries reveals an asymmetry; dense intermediate-position queries reveal a general failure of position-indexed retrieval; and our interventions demonstrate the failure is recoverable. Our task is also distinguished by *destructive* updates (earlier values are overwritten, not just earlier in context) and a regime-dependent bias whose direction flips with stream load rather than tracing a single U-shape.

Fine-tuning surfaces latent capacity. Recent work has shown that capabilities not visible in pretrained models can be surfaced with minimal supervision. Zhou et al. (2023) demonstrate that 1,000 curated dialogues are sufficient to teach a pretrained model to follow chat-style instructions, attributing this to pretraining having already done the work of knowledge acquisition. Sun and

Dredze (2025) make a more general claim across 18 datasets: pretraining improves models in latent ways only visible after fine-tuning. The methods we use — low-rank adaptation (Hu et al., 2022) and the observation that fine-tuning lives in a low intrinsic dimension (Aghajanyan et al., 2021) — are by now standard. Our contribution against this background is the concrete instance with mechanism: a specific capability (in-context update retrieval) that pretrained LLMs systematically fail at, surfaced by an 18,000-example LoRA across held-out grid cells, datasets, and ∼10× context lengths, with the recovery traced mechanistically to upper-middle-layer attention heads ⟨**NOTE:** post-LoRA pre/post comparison pending⟩. The distinction from LIMA is style/alignment versus mechanistic capability recovery; the distinction from Amuro & Char is general claim versus concrete instance with mechanism.

Mechanism and pretraining dynamics. Our mechanism and pretraining-dynamics analyses build on two threads. Mechanistic interpretability has identified circuits for specific capabilities — induction heads forming during pretraining as a phase transition (Olsson et al., 2022), indirect-object identification circuits (Wang et al., 2023), automated circuit discovery (Conmy et al., 2023), and interpretable algorithms in successful state trackers (Li et al., 2025). Closest to our setting, Mishra (2025) report that 62% of attention heads shift function during fine-tuning on summarization, with middle layers exhibiting the most dramatic change; we apply the same approach to a mechanistic capability rather than a style task. On the pretraining-capability side, Wei et al. (2022) characterize abilities that appear at scale and Schaeffer et al. (2023) argue these emergences are metric artifacts; our work is a *non-emergence* question — the success of this skill does not emerge at any scale we test. Allen-Zhu and Li (2023) show that knowledge can be memorized during pretraining but not extractable without targeted augmentation, paralleling our latent-capacity claim: the architecture stores the information; pretraining does not develop the retrieval pathway by default.

3 Task and Setup

Task. Let $\mathcal{K} = \{k_1, \ldots, k_K\}$ be a set of keys and let $V_i = (v_{i,1}, \ldots, v_{i,N})$ be an ordered update sequence for key k_i. The input stream σ is a random interleaving of all $K \times N$ pairs $\{(k_i, v_{i,j})\}$, subject to the constraint that no two consecutive entries share the same key. The model receives σ and a target key k_t and is asked one of three queries: **FVQ** (first-value, expected answer $v_{t,1}$); **CVQ** (current-value, expected answer $v_{t,N}$); or **IVQ** (intermediate-value, expected answer $v_{t,k}$ for some $1 < k < N$). Figure 1 shows a small instance. The intermediate-value query is used only in the dense-retrieval analysis (§5; Appendix I).

Datasets. **Arbitrary-Single (ARB)** uses a pool of 2,300 single-token English words across 46 semantic categories; single-token values keep cross-entropy and probing computations clean and make it the dataset for the mechanistic analysis (§7). **Semantic-Multi (SEM)** uses real category members as multi-token values across the same 46 categories (2,403 entries total) and is our primary behavioural dataset — semantic coherence between key and value limits template-matching shortcuts. Full construction details are in Appendix A.

Grid and trials. We evaluate models on a unified grid $K \in \{2, 3, 5, 7, 10, 15, 20, 25, 30\}$, $N \in \{5, 7, 10, 15, 20, 30, 50, 75, 100\}$, with up to 200 trials per (K, N) cell per query type. Cells where the prompt exceeds a model's context limit or where the dataset pool cannot supply N distinct values per key are excluded (specifics in Appendix A).

Metric. For each (K, N) cell we report FVQ and CVQ accuracy as string-match proportions (matching rule in Appendix B) with Wilson 95% confidence intervals (Wilson, 1927). The primary metric is the **gap** Δ = FVQ accuracy − CVQ accuracy; we call a gap **substantial** when $|\Delta| > 0.05$. All inferences use greedy decoding (temperature 0) to eliminate sampling variance.

Models. We evaluate **11 open-weight models** from three major families — Qwen2.5 (0.5B–3B; instruct and base), Qwen3.5 (0.8B–9B; instruct), Gemma-3 (270M–4B; instruct) — and **6 proprietary frontier models** from OpenAI (GPT-4.1, GPT-4.1-mini), Anthropic (Claude-4.5-Haiku, Claude-4.5-Sonnet), and Google (Gemini-2.5-Flash, Gemini-2.5-Pro). Three additional smaller models — TinyLlama-1.1B, StableLM-2-1.6B, Pythia-410M — appear only in the appendix. The full model list is in Appendix C; per-cell behavioural results are in Appendix D.

4 Pretraining Fails at In-Context Update Retrieval

Every model we evaluate exhibits a substantial FVQ–CVQ gap. At the representative cells in Table 1, gaps range from +6 to +53 percentage points, with the majority exceeding +20 pp. Per-cell evidence across the full (K, N) grid for all 20 models is in Appendix D. Figure 2 shows the trajectory in two open-weight models under three axis scans, including one that reaches the reversal regime.

The gap takes two shapes. In most cells of Table 1 — the **primacy regime** — FVQ exceeds CVQ: the model anchors to the first value bound to the key and fails to overwrite. In several smaller open-weight models at higher (K, N), the gap inverts to a **reversal regime**: CVQ exceeds FVQ, but both queries stay well below ceiling, with CVQ accuracy in the 50–65% range. Figure 2's third panel column shows the reversal explicitly for Qwen2.5-3B-Instruct, while Gemma-3-4b-it stays in the primacy regime throughout. Neither regime equals correct retrieval; both are deviations from the answer the prompt asks for. We trace the mechanism underlying the regime split in §7.

Intermediate-position queries ($1 < k < N$) collapse uniformly across all five open-weight models in our ucurve sweep (Qwen2.5-3B-Instruct, Qwen3.5-2B, Qwen3.5-4B, Qwen3.5-9B, Gemma-3-4b-it) under the default Plain format: at $K{=}10, N{=}50$, accuracy drops below 5% within two positions of the stream start and remains there (Appendix F, multi-model IVQ figure, left panel). Three of the six proprietary models we evaluate show the same pattern (GPT-4.1, GPT-4.1-mini, Claude-4.5-Haiku); Claude-4.5-Sonnet shows U-shaped recovery and Gemini-2.5-Flash a partial dip; Gemini-2.5-Pro is the single model that maintains near-ceiling accuracy across all positions, indicating the capability is achievable but not developed in the majority of models we test. A deeper per-cell dive for Qwen2.5-3B-Instruct with the LoRA recovery panel is in Appendix I (cited again in §5). The FVQ–CVQ asymmetry measured above is therefore one slice of a broader failure of position-indexed retrieval — present in 8 of the 11 models in this sweep (all five open-weight, plus three of six proprietary).

The rest of the paper asks two questions about this failure: whether it is recoverable from the architecture's existing capacity (§5), and when during pretraining and post-training the failure develops (§6).

Model	FVQ	CVQ	Gap	% Ctx
Proprietary models, K=20, N=50				
GPT-4.1	1.00	0.71	+0.29	0.7%
GPT-4.1-mini	1.00	0.74	+0.26	0.7%
Claude-4.5-Haiku	1.00	0.53	+0.47	3.3%
Claude-4.5-Sonnet	1.00	0.74	+0.26	0.7%
Gemini-2.5-Flash	1.00	0.54	+0.46	0.6%
Gemini-2.5-Pro	0.90	0.77	+0.13	0.7%
Open-weight models, K=5, N=30				
Qwen2.5-3B-Instruct	0.61	0.55	+0.06	3.1%
Qwen3.5-2B	0.62	0.48	+0.14	0.4%
Qwen3.5-4B	0.86	0.64	+0.22	0.4%
Qwen3.5-9B	0.86	0.79	+0.07	0.4%
Gemma-3-1b-it	0.55	0.02	+0.53	3.1%
Gemma-3-4b-it	0.97	0.68	+0.29	0.8%

Table 1: FVQ and CVQ accuracy on Semantic-Multi at the representative cell per block: $K{=}20, N{=}50$ for proprietary, $K{=}5, N{=}30$ for open-weight (the smallest cell at which every model in the block exhibits gap $> +0.05$). Up to 200 trials per cell; Wilson 95% CIs in Appendix D. **% Ctx** is the prompt token count as a fraction of the model's advertised context window; see Appendix D for the tokeniser and per-row window sources. No model uses more than **3.3%** of its context at the cell it fails on.

5 Two Recoveries: Format and LoRA

5.1 Format intervention

If the failure measured in §4 is about *locating* the relevant entries in the stream, making positions explicit at the surface should make the lookup easier. We test this with four prompt formats applied to the same key-value stream: **Plain** (the default condition used in §4, no positional markers); **Labeled** (each entry tagged with an explicit update index); **Block** (entries grouped under per-round headers); and **Landmark** (block layout with --- separating rounds, no numbers). Worked examples for each format are in Appendix B. We report results at $K{=}10, N{=}50$ — the hardest cell in the format sweep — across 11 models (5 open-weight, 6 proprietary) in Table 2.

Block format eliminates the FVQ–CVQ gap across every open-weight model where we measure it (Table 2). The recovery is genuine, not equalisation: under Block both FVQ and CVQ sit near ceiling for every open-weight model where we have the data, whereas under Plain CVQ collapses while FVQ stays at ceiling. Proprietary models follow the same pattern at smaller magnitudes; Gemini-2.5-Pro is the only model at ceiling on both queries under Plain, consistent with its non-collapse in

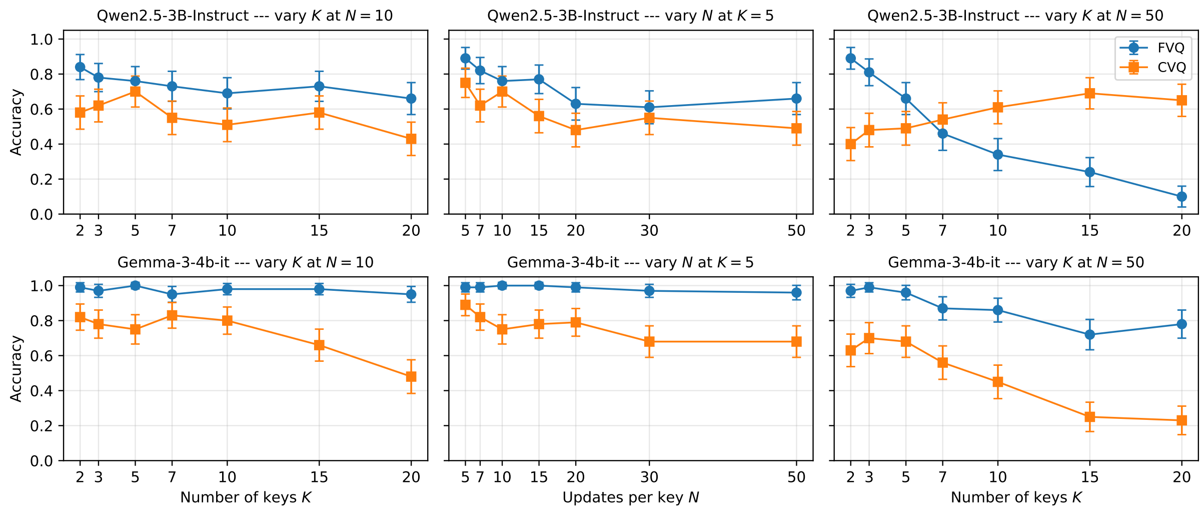

Figure 2: FVQ and CVQ accuracy on Semantic-Multi for Qwen2.5-3B-Instruct (top) and Gemma-3-4b-it (bottom). Columns vary K at N=10 (left), N at K=5 (middle), and K at N=50 (right). The right panel shows Qwen2.5-3B-Instruct crossing into the reversal regime at $K \approx 7$ while Gemma-3-4b-it stays in the primacy regime throughout. Error bars are Wilson 95% CIs.

Model	**Plain**	**Labeled**	**Block**	**Landmark**
Open-weight models				
Qwen2.5-3B-Instruct	0.49 / 0.23	0.34 / 0.39	0.84 / 0.92	0.71 / 0.27
Qwen3.5-2B	0.95 / 0.01	0.85 / 0.93	1.00 / 0.98	0.80 / 0.07
Qwen3.5-4B	0.99 / 0.00	0.93 / 0.97	1.00 / 1.00	0.99 / 0.47
Qwen3.5-9B	1.00 / 0.04	—	—	—
Gemma-3-4b-it	0.92 / 0.01	0.39 / 0.47	0.99 / 0.89	0.92 / 0.00
Proprietary models				
GPT-4.1	1.00 / 0.69	1.00 / 0.96	1.00 / 0.98	0.99 / 0.99
GPT-4.1-mini	1.00 / 0.77	1.00 / 0.94	0.99 / 0.99	0.78 / 1.00
Claude-4.5-Haiku	1.00 / 0.43	1.00 / 0.95	1.00 / 0.99	0.91 / 0.90
Claude-4.5-Sonnet	0.99 / 0.80	1.00 / 0.81	1.00 / 1.00	0.96 / 0.99
Gemini-2.5-Flash	1.00 / 0.76	1.00 / 0.98	1.00 / 1.00	1.00 / 0.95
Gemini-2.5-Pro	1.00 / 1.00	1.00 / 1.00	1.00 / 1.00	0.54 / 0.99

Table 2: Format-intervention results on Semantic-Multi at K=10, N=50. Each cell shows **FVQ / CVQ accuracy** under that format (e.g., "0.99 / 0.23" = FVQ 0.99, CVQ 0.23). The gap Δ is FVQ−CVQ; cells where the two numbers are close indicate no gap. Plain is the default condition used in §4. Up to 200 trials per (model, format) cell.

§4. ⟨**NOTE:** Qwen3.5-9B has only Plain-format data at K=10, N=50; the remaining three formats were not run.⟩

Labeled and **Landmark** produce mixed outcomes. Labeled (per-entry numbering) closes the gap in some open-weight models by *degrading FVQ* rather than restoring CVQ — in Qwen2.5-3B-Instruct both queries land near 0.4 (Table 2), shifting the failure rather than removing it. Landmark (round markers, no numbers) recovers the proprietary set but leaves open-weight models near their Plain-format failure (Gemma-3-4b-it stays at FVQ≈0.97 / CVQ≈0.05). Across the three intervention formats only *Block* — round-grouping combined with explicit round numbers — reliably restores both queries in the open-weight set; neither round structure nor per-entry numbering alone is enough.

The format intervention demonstrates that the retrieval circuit exists in the pretrained model: given the right surface cue at inference time, the model already has what it needs to retrieve the current value of an in-context variable. Pretraining does not, by default, dispatch this circuit from the Plain key-value format. The same capability is also recoverable from within the model, via targeted fine-tuning, which we show next.

5.2 LoRA intervention

A small targeted fine-tune recovers the capability without any inference-time change to the prompt format. We fit a LoRA adapter (Hu et al., 2022) (rank 16, attention-only $\{Q, K, V, O\}$, two epochs) to Qwen2.5-3B-Instruct on 18,000 examples drawn from a deliberately small slice of our grid: $K \in \{2, 3, 5, 10\}$, $N \in \{5, 10, 15, 20\}$, sampled across 35 of 46 categories. The adapter adds 7.4M trainable parameters — 0.24% of model weights. We evaluate on a held-out grid that extends to K=30 and N=75, on the 11 categories never seen during training, and cross-distribution on Semantic-Multi. Full training configuration is in Appendix H.

On the 28 held-out Arbitrary-Single cells, both FVQ and CVQ accuracy reach 93–100% post-adapter (Table 3). The held-out cells extend roughly 10× beyond the training grid in stream length; the LoRA holds across this extrapolation without per-cell tuning. The recovery is generalisation, not memorisation: the adapter has learned a task pattern, not the specific cells in its training set.

The same adapter, trained only on Arbitrary-Single, transfers to Semantic-Multi without modification (Table 4). At four held-out SEM cells, both queries reach 100% post-LoRA — including cells in the deep reversal regime with baseline gap up to -0.45. The LoRA has not learned the lexical pattern of one dataset; it has learned to do position-indexed retrieval.

The dense intermediate-position queries that collapsed under the base model in §4 are also recovered. The LoRA-tuned model reaches near-ceiling at every queried position across six cells (Figure 3; Appendix I). This rules out the alternative that the LoRA merely amplifies a recency-attending circuit: such an account would recover CVQ at the expense of intermediate-position accuracy, but the data shows simultaneous recovery across positions.

Two pre-registered controls test the specificity of the recovery. A family-control LoRA on Gemma-3-4b-it with identical configuration probes whether the recovery is architecture-agnostic ⟨**NOTE:** family-control experiment pending⟩. A negative-control LoRA trained on arithmetic word problems with the same hyperparameters probes whether targeted task supervision (rather than any LoRA training) is the relevant cause ⟨**NOTE:** arithmetic-control experiment pending⟩; we expect arithmetic LoRA to leave the FVQ–CVQ gap unchanged.

The format and LoRA interventions converge: both close the FVQ–CVQ gap and restore intermediate-position retrieval. The first does so at inference time without gradients; the second with a fraction of a percent of the model's parameters.

(K, N)	**Base gap**	**+ LoRA gap**
$(7, 30)$	+12%	+0%
$(7, 50)$	+6%	+4%
$(10, 30)$	−5%	+0%
$(10, 50)$	−8%	+0%
$(15, 20)$	−26%	+2%
$(15, 30)$	−25%	+2%
$(15, 50)$	−41%	+0%
$(20, 30)$	−30%	+2%
$(20, 50)$	−54%	+4%
$(20, 75)$	−50%	+4%
$(25, 30)$	−44%	+2%
$(25, 50)$	−38%	+2%
$(25, 75)$	−54%	+7%
$(30, 30)$	−48%	+2%
$(30, 50)$	−50%	+6%
$(30, 75)$	−55%	+6%

Table 3: LoRA intervention on Qwen2.5-3B-Instruct, evaluated on held-out Arbitrary-Single cells. The adapter is trained on a small grid ($K \leq 10, N \leq 20$); we report base vs +LoRA gap on the 16 test-grid cells extending to K=30, N=75. Across every held-out cell, both FVQ and CVQ accuracy reach 93–100 % post-adapter; full FVQ and CVQ breakdowns are in Table 14 (Appendix H).

	Base			**+ LoRA (ARB-trained)**		
(K, N)	FVQ	CVQ	gap	FVQ	CVQ	gap
$(7, 20)$	60%	55%	+5%	100%	100%	+0%
$(15, 20)$	25%	55%	−30%	100%	100%	+0%
$(15, 30)$	35%	80%	−45%	100%	100%	+0%
$(20, 30)$	25%	65%	−40%	100%	100%	+0%

Table 4: Cross-distribution transfer: the same LoRA trained only on Arbitrary-Single restores accuracy to ceiling on Semantic-Multi (multi-token semantic values) without modification. Four held-out (K, N) cells; all four go from baseline reversal regime to gap 0%.

The capability is therefore present in the pretrained architecture but not dispatched by default. §6 asks when in training this dispatching failure develops; §7 asks what changes mechanistically when LoRA recovers it.

6 When Does the Failure Develop?

The recovery in §5 shows the capability is in the architecture. We ask when during training the failure to dispatch this capability develops. We track the FVQ–CVQ gap across 32 published pretraining checkpoints of SmolLM3-3B (Hugging Face Smol Models Research Team, 2025) (sampled across all three documented training stages, out of 118 released) plus the five documented post-training stages — mid-training, SFT, APO, LC-expert, and the released final model. We evaluate

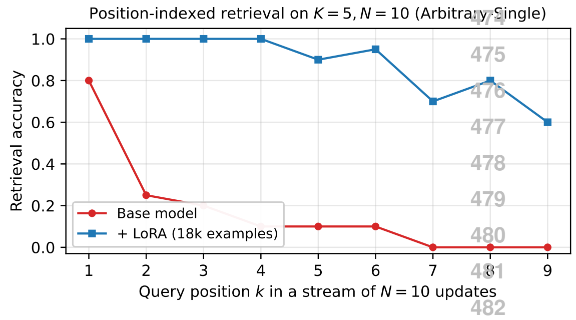

Figure 3: Per-position retrieval accuracy at $K{=}5, N{=}10$ on Arbitrary-Single, for positions $k \in [1, N-1]$ (the experiment substitutes a "last value" query at $k{=}N$, so we omit that point as it is operationally CVQ rather than IVQ). The base model recovers position 1 reliably and collapses on intermediate positions; the LoRA-tuned model is at or near ceiling at every queried position.

on the Arbitrary-Single grid (§3), partitioning the 32-cell test set into three exhaustive stream-load regimes: **low** ($K{\le}3$, any N; 16 cells), **medium** ($K{\ge}5, N{<}20$; 12 cells), and **high** ($K{\ge}5, N{\ge}20$; 4 cells). This load-based partition complements the outcome-based regime language of §4: low- and medium-load cells map onto the primacy regime, while the reversal regime, where it appears, lives in the high-load cells. SmolLM2-1.7B (41 pretraining checkpoints plus the released final) shows the same regime-dependent pattern under the same partition; its trajectory is in Appendix G.

Figure 4 shows the gap remains directionally biased at every checkpoint, in every regime. Low-load cells stay clearly in the primacy direction (mean gap +0.2 to +0.7) across the entire trajectory. Medium-load cells stay weakly positive but rarely touch zero. High-load cells oscillate in direction across pretraining but never converge to zero. No regime stabilises at zero at any documented checkpoint.

None of the five documented post-training stages closes the gap in any regime. The bias the model carries out of pretraining survives mid-training, SFT, APO, LC-expert, and the released final model. SmolLM2-1.7B shows the same picture: the gap is direction-biased at the earliest released checkpoint (step 125k) and at every subsequent checkpoint through the published four-stage curriculum (Ben Allal et al., 2025), including the released final (Appendix G).

The failure is therefore not introduced by alignment, not by supervised fine-tuning, not by mid-training reasoning data, and not by any late-pretraining curriculum shift. The same models succeed at multi-step reasoning, code synthesis, and long-context retrieval at scales of difficulty that exceed this task. The capability is recoverable by a small adapter (§5.2); pretraining does not develop it. We characterise what changes inside the model when LoRA recovers the capability in §7 and return to what this implies about training signals in §8.

7 Mechanism: What LoRA Changes

The recovery in §5.2 shows the capability is present in the pretrained architecture; §6 shows it remains undispatched through every documented training stage. We now ask: what changes inside the model when an 18,000-example LoRA dispatches it? Four mechanistic methods — probing, logit lens, attention routing, and causal ablation — converge on the same answer: the failure is a routing problem, not a representation problem. The substrate was already present in pretraining; LoRA installs the missing connection in 15 attention heads of layers 30–33. All measurements use Qwen2.5-3B-Instruct on Arbitrary-Single at $K{=}2, N{=}5$ unless noted otherwise; layer indexing is 0-based (L0–L35). Table 5 reports bootstrap 95% CIs on the focal deltas; Figure 5 shows the per-layer trajectories and the spatial localisation of the rewiring.

7.1 The substrate already exists in baseline

A linear probe trained on the residual stream to distinguish the RI from the PI prompt reaches 100% accuracy from layer 6 onward in *both* the baseline and LoRA models; LoRA adds nothing to this signal. The pretrained model already knows which question it is being asked. A second probe, trained instead to predict whether the model *will get RI right*, reaches 81% accuracy at L33 in baseline — the model also internally tracks RI correctness.

What the baseline does *not* encode is whether it will get PI right. The PI-correctness probe at L27–L33 sits at chance in baseline (53–58%) while the corresponding RI probe is at 77–81%. Logit lens reveals the parallel pattern at the token level: at L32 of the baseline, $\Pr(v_{\text{last}}){=}0.34$ — the answer surfaces briefly — then decays to $\Pr(v_{\text{last}}){=}0.25$ at L35. *Found then suppressed.* At a harder cell ($K{=}2, N{=}50$) the answer barely surfaces at all ($\Pr(v_{\text{last}}){=}0.02$ at L32, 0.11 at L35), tracking the behavioural failure. The pretrained model identifies

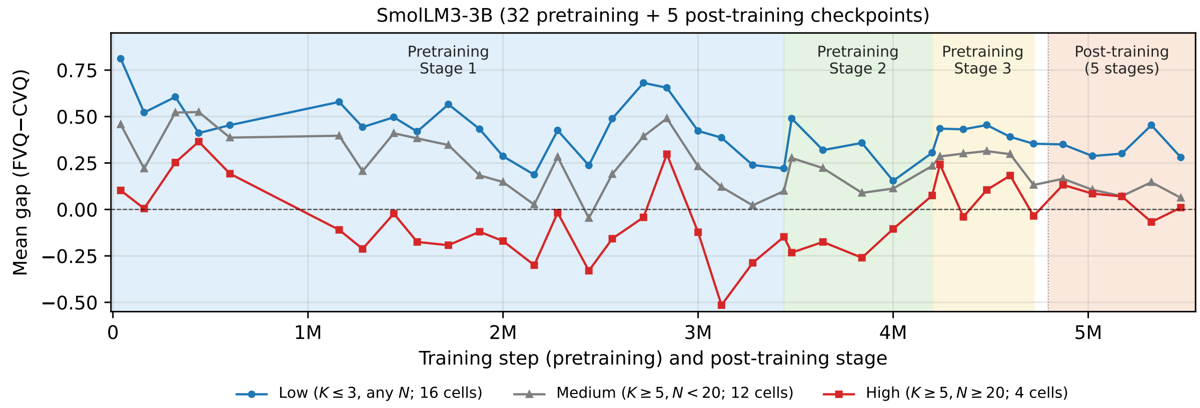

Figure 4: Mean FVQ−CVQ gap on Arbitrary-Single across the SmolLM3-3B (Hugging Face Smol Models Research Team, 2025) training trajectory, partitioned into three exhaustive load regimes: **Low** ($K{\leq}3$, any N; 16 cells, blue circles), **Medium** ($K{\geq}5, N{<}20$; 12 cells, grey triangles), **High** ($K{\geq}5, N{\geq}20$; 4 cells, red squares). Coloured bands mark the three documented pretraining stages. The pink band on the right contains, in order, the five documented post-training stages: mid-training, SFT, APO, LC-expert, final; within this band the x-axis is categorical rather than step-indexed. SmolLM2-1.7B shows the same regime structure across its pretraining curriculum and final model; see Appendix G, Figure 9.

the query, partially computes the answer, and has an intact readout machinery in layers 34–35 (their attention patterns are nearly identical pre- and post-LoRA, see Figure 5b). It fails because the partial answer does not propagate.

7.2 LoRA amplifies a partial v_{last} promoter circuit in layers 30–33

Attention routing locates the change precisely. We measure, per head, the probability of attending from the generation token to the round containing v_{last} under the PI prompt. Fifteen heads in the L30–L33 band, identified in Figure 5b, gain substantial v_{last} attention: pre-LoRA values of 0.05–0.23 rise to 0.60–0.86 post-LoRA — a 4–12× increase per head. The largest single-head shifts are L32H3 ($0.11 \to 0.78$), L32H7 ($0.13 \to 0.78$), L31H12 ($0.17 \to 0.81$), and L31H15 ($0.23 \to 0.86$). Layers 0–20 attention patterns are essentially unchanged (small negative shifts of -0.04 to -0.06 on a few heads); layers 34–35 attention patterns are also unchanged. The change is spatially confined to a four-layer band in the upper-middle of the stack.

Importantly, the pre-LoRA values are non-zero. The heads were already *partial* v_{last} attenders before any LoRA training, and (as we show in §7.3) they already carried causal v_{last} signal in baseline. LoRA does not install these heads from scratch; it amplifies a partial pre-existing circuit by 4–12×.

The token-level consequence is immediate. Where baseline L32 has $\Pr(v_{\text{last}}){=}0.34$, post-LoRA L32 has 0.99 at $K{=}2, N{=}5$, and 0.62 at the harder $K{=}2, N{=}50$ cell. By L33–L35 the probability saturates to 1.0 even at the harder cell. The representational consequence is parallel: the PI-correctness probe at L27–L33 rises from chance to 92–94% (bootstrap 95% CI $\Delta = +0.36$ $[+0.21, +0.50]$ at L33), installed in the same late layers that already carried the RI-correctness signal.

A naive alternative would be that LoRA *removes* the late-layer suppression that attenuated v_{last} in baseline. Stage 3 causal ablation rules this out. Ablating the five baseline suppressor heads (L26H3, L27H3, L29H3, L29H4, L30H3) in the LoRA model still produces a measurable release of v_{last} at L31 ($\Delta{=}{+}0.150$, *larger* than the $+0.078$ release seen when the same heads are ablated in baseline). The suppressors keep firing post-LoRA, and they have more signal to suppress. LoRA does not remove them; it strengthens a parallel promoter pathway that overpowers them.

7.3 Two LoRA changes: amplification and downstream redundancy

To distinguish what LoRA *installs* from what it *amplifies*, we ablate the eight highest-attribution promoter heads in both the baseline and the LoRA model and compare the resulting trajectories. Table 6 summarises the four conditions; the paired within-run Δ at L32 is the trustworthy statistic (cross-run normal trajectories differ because trial seeds are not pinned across runs).

Two findings emerge. First, ablating the eight heads in the *baseline* model drops L32 $\Pr(v_{\text{last}})$ by -0.38, *comparable in magnitude to the* -0.28 *drop in the LoRA model*. The heads are not LoRA-installed; they already carry partial causal v_{last} signal in pretraining, consistent with their non-zero pre-LoRA $\Pr(\text{attend to } v_{\text{last}})$ in attention routing. The cleanest description of what LoRA does to these heads is therefore *amplification*: it increases the gain of a circuit that was already partially functional.

Second, the downstream propagation of the perturbation differs sharply between baseline and LoRA. In baseline, ablating the L30–L33 promoters propagates through readout: L33 $\Delta = -0.13$, L35 $\Delta = -0.09$. The final-layer $\Pr(v_{\text{last}})$ retains a measurable hit. In the LoRA model, the same ablation is absorbed by L33+: L33 $\Delta = +0.00$, L35 $\Delta = +0.00$. The downstream layers have gained compensatory pathways that re-amplify v_{last} when the upstream signal is partially knocked out. This redundancy is genuinely new — it does not pre-exist in the baseline architecture in any form we can detect by ablation.

Synthesis. The four methods form a closed causal chain: attention heads in L30–L33 attend to v_{last} (attention routing) $\rightarrow$ v_{last} builds in the residual stream at L32 (logit lens) $\rightarrow$ a PI-correctness feature becomes linearly decodable at L27–L33 (probing) $\rightarrow$ behaviour recovers (§5.2). LoRA does two things in this chain. It *amplifies* a partial v_{last}-attending circuit that was already present in pre-training (L30–L32 heads attend 4–12× more than they did pre-training, and carried partial causal signal even pre-training). It also *installs* downstream redundancy at L33+, which absorbs upstream perturbations that would have propagated through the baseline readout. The substrate was therefore even more complete than a first reading suggests: not only the v_{last} representation, condition discrimination, and readout, but also a partial promoter circuit were all present pre-training. The pretraining failure is a gain and robustness failure — the partial circuit fires too weakly to win the tug-of-war against suppression, and the readout has no redundancy to compensate when the upstream signal is incomplete. 18 000 LoRA examples turn up the gain and install error-correcting pathways. We discuss what this implies for the broader latent capability claim in §8.

8 Discussion

⟨**NOTE:** Three implications: scaling won't fix this trivially; LM loss has a structural blind spot; characterise–recover–locate as a methodology.⟩

Limitations

We highlight four directions where the present study could be extended.

Real-world stateful benchmarks. Our task is a controlled synthetic instance of in-context state tracking. Connecting the gap we measure to application-level failures — multi-turn dialogue degradation (Laban et al., 2025) and entity state-tracking errors (Kim and Schuster, 2023; Kim et al., 2024; Rezaee et al., 2025) — is a natural next step.

Scale and language coverage. All experiments are in English; the training-dynamics analysis uses SmolLM2 and SmolLM3 at $\leq$ 3B parameters — the public checkpoint releases that permit per-step evaluation at this granularity. Extending the trajectory study to $\geq$ 7B models, non-Transformer architectures, and other languages is future work.

Prose-context generalisation. We explored a prose-narrative variant of the task built on a Dota 2 match-commentary corpus but found it difficult to construct in a way that disentangles the position-indexed retrieval failure we measure from natural event-ordering structure in the narration. Clean narrative testing of the same skill is open.

Predicting the reversal regime. Some models cross from primacy into reversal at high stream load (Qwen-family); others stay in primacy throughout (Gemma-family). What model property — depth, attention sparsity, training-data composition, or otherwise — predicts this regime split is an open question.

Ethical Considerations

Our experiments use synthetic key–value streams (Arbitrary-Single, Semantic-Multi) constructed from public English vocabulary lists; no human subjects are involved and no personally identifying information is collected. Open-weight models are used under their respective Apache or Gemma licenses; proprietary models are queried through commercial APIs under standard terms of service. The LoRA fine-tune trains 7.4M parameters for ∼1.5 hours on a single L40S GPU. Our results

Method	Quantity	Baseline	+LoRA	Δ [95% CI]
Probe	PI-correctness, L33	0.58	0.94	+0.36 [+0.21, +0.50]
Probe	PI-correctness, L32	0.65	0.94	+0.29 [+0.22, +0.36]
Probe	PI-correctness, L30	0.57	0.84	+0.28 [+0.19, +0.37]
Probe	RI-correctness, L33	0.81	0.93	+0.12 [+0.05, +0.19]
Logit lens	$\Pr(v_{\text{last}})$, L32, K=2, N=5	0.34	0.99	+0.65 [+0.55, +0.73]
Logit lens	$\Pr(v_{\text{last}})$, L32, K=2, N=10	0.21	0.93	+0.73 [+0.63, +0.81]
Logit lens	$\Pr(v_{\text{last}})$, L32, K=2, N=50	0.02	0.62	+0.60 [+0.50, +0.69]
Logit lens	$\Pr(v_{\text{last}})$, L35, K=2, N=50	0.11	0.90	+0.79 [+0.71, +0.87]
Attention routing	$\Pr(\text{attend } v_{\text{last}})$, L32H3	0.11	0.78	+0.68 [+0.65, +0.71]
Attention routing	$\Pr(\text{attend } v_{\text{last}})$, L31H12	0.16	0.81	+0.66 [+0.60, +0.71]
Attention routing	$\Pr(\text{attend } v_{\text{last}})$, L32H7	0.12	0.78	+0.65 [+0.62, +0.68]
Attention routing	$\Pr(\text{attend } v_{\text{last}})$, L31H15	0.23	0.86	+0.64 [+0.57, +0.70]

Table 5: Cross-method bootstrap 95% CIs on the focal mechanistic deltas at the L30–L33 band, baseline vs +LoRA. Probing and attention routing aggregate across the relevant heads or layers; logit-lens rows report $\Pr(v_{\text{last}})$ at L32 for the focal (K, N) cells in our analysis.

Ablation condition	Δ at L31	Δ at L32	Δ at L33	Δ at L35
Baseline + 5 baseline-suppressor heads	+0.078	−0.009	−0.058	+0.015
+LoRA + 5 baseline-suppressor heads	+0.150	−0.001	+0.000	+0.000
+LoRA + 8 LoRA-promoter heads	+0.050	−0.278	+0.000	+0.000
Baseline + 8 LoRA-promoter heads (control)	+0.003	**−0.377**	−0.133	−0.088

Table 6: Stage 3C four-way causal-ablation summary. Each row reports the paired within-run Δ= ablated − normal at four representative layers. The bold row is the new baseline control (K=2, N=5, 50 trials per cell, PI condition): ablating the same eight LoRA-promoter heads in the *baseline* model produces a comparable L32 drop (−0.377 vs −0.278 in the LoRA model), establishing that the heads were partial v_{last} carriers pre-LoRA. The downstream columns (L33, L35) tell the complementary story: baseline ablation propagates through readout (−0.133, −0.088); the LoRA model absorbs the same perturbation at L33+ (+0.000, +0.000), revealing the new downstream redundancy.

document a deployment-relevant failure mode — LLMs should not be relied upon to track the current state of repeatedly updated in-context variables without explicit positional cues or targeted fine-tuning — and we hope this informs responsible deployment of in-context state-tracking systems.

References

Armen Aghajanyan, Sonal Gupta, and Luke Zettlemoyer. 2021. Intrinsic dimensionality explains the effectiveness of language model fine-tuning. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics (ACL)*.

Zeyuan Allen-Zhu and Yuanzhi Li. 2023. Physics of language models: Part 3.1, knowledge storage and extraction. *arXiv preprint arXiv:2309.14316*.

Loubna Ben Allal, Anton Lozhkov, Elie Bakouch, Jacob Blakeney, Guilherme Penedo, Hynek Kydlíček, Tinghao Yu, Yuwei Yang, Joffrey Patiny, Christopher Cohen, Carolyn Yan, Mathilde Maillard, Agustín Martín Blázquez, Lewis Tunstall, Andrés Marafioti, Vaibhav Šakar, Leandro von Werra, and Thomas Wolf. 2025. SmolLM2: When smol goes big — data-centric training of a small language model. *arXiv preprint arXiv:2502.02737*.

Arthur Conmy, Augustine Mavor-Parker, Aengus Lynch, Stefan Heimersheim, and Adrià Garriga-Alonso. 2023. Towards automated circuit discovery for mechanistic interpretability. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Thomas Fagnou, Brian Lester, Mingqiu Zhao, and Chris Dyer. 2024. Chain and causal attention for efficient entity tracking. In *Proceedings of EMNLP*.

Michael Hahn. 2020. Theoretical limitations of self-attention in neural sequence models. In *Transactions of the Association for Computational Linguistics*, volume 8, pages 156–171.

Cheng-Ping Hsieh, Simeng Sun, Samuel Kriman, Shantanu Acharya, Dima Rekesh, Fei Jia, Yang Zhang, and Boris Ginsburg. 2024. RULER: What's the real context window size of your long-context language models? *arXiv preprint arXiv:2404.06654*.

Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2022. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*.

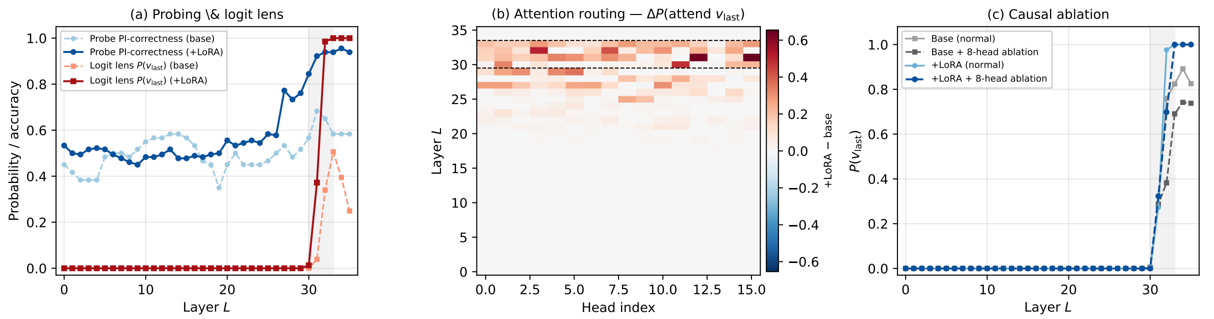

Figure 5: Mechanism summary for Qwen2.5-3B-Instruct, $K{=}2$, $N{=}5$ unless noted. **(a)** Per-layer $\Pr(v_{\text{last}})$ at the answer position (logit lens) and PI-correctness probe accuracy, baseline vs +LoRA. Both signals converge on the L27–L33 band, shaded for reference. **(b)** Per-head Δ in $\Pr(\text{attend to } v_{\text{last}})$ from attention routing under PI, across all 36 layers $\times$ 16 heads. The bright band at L30–L33 (dashed outline) contains the 15 promoter heads identified in §7.2. **(c)** Stage 3C causal ablation: ablating the eight highest-attribution promoter heads drops $\Pr(v_{\text{last}})$ sharply at L32 in *both* the baseline and LoRA models, showing the heads were already partial v_{last} carriers pre-training. Baseline ablation continues to drop at L33–L35; the same perturbation in the LoRA model is absorbed by L33+ redundancy and recovers to $\Pr(v_{\text{last}}){\approx}1.0$ at the final layer.

Hugging Face Smol Models Research Team. 2025. SmolLM3: smol, multilingual, long-context reasoner. https://huggingface.co/blog/smollm3. Blog post; model: https://huggingface.co/HuggingFaceTB/SmolLM3-3B.

Najoung Kim and Sebastian Schuster. 2023. Entity tracking in language models. In *Proceedings of ACL*.

Najoung Kim, Sebastian Schuster, and Shubham Toshniwal. 2024. Code pretraining improves entity tracking abilities of language models. *arXiv preprint arXiv:2405.21068*.

Philippe Laban, Hiroaki Hayashi, Yingbo Zhou, and Jennifer Neville. 2025. LLMs get lost in multi-turn conversation. *arXiv preprint arXiv:2505.06120*.

Belinda Z. Li, Zifan Carl Guo, and Jacob Andreas. 2025. (how) do language models track state? *arXiv preprint arXiv:2503.02854*.

Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. 2024. Lost in the middle: How language models use long contexts. In *Transactions of the Association for Computational Linguistics*, volume 12, pages 157–173.

William Merrill and Ashish Sabharwal. 2023. A logic for expressing log-precision transformers. In *Advances in Neural Information Processing Systems*.

Anurag Mishra. 2025. Mechanistic interpretability of GPT-like models on summarization tasks. *arXiv preprint arXiv:2505.17073*.

Catherine Olsson, Nelson Elhage, Neel Nanda, Nicholas Joseph, Nova DasSarma, Tom Henighan, Ben Mann, Amanda Askell, Yuntao Bai, Anna Chen, Tom Conerly, Dawn Drain, Deep Ganguli, Zac Hatfield-Dodds, Danny Hernandez, Scott Johnston, Andy Jones, Jackson Kernion, Liane Lovitt, and 7 others. 2022. In-context learning and induction heads. *Transformer Circuits Thread*.

Qwen Team. 2025. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*.

Kiamehr Rezaee, Jose Camacho-Collados, and Mohammad Taher Pilehvar. 2025. Exploring state tracking capabilities of large language models. *arXiv preprint arXiv:2511.10457*.

Rylan Schaeffer, Brando Miranda, and Sanmi Koyejo. 2023. Are emergent abilities of large language models a mirage? In *Advances in Neural Information Processing Systems (NeurIPS)*.

Melanie Sclar, Yejin Choi, Yulia Tsvetkov, and Alane Suhr. 2024. Quantifying language models' sensitivity to spurious features in prompt design or: How I learned to start worrying about prompt formatting. In *Proceedings of ICLR*.

Kaiser Sun and Mark Dredze. 2025. Amuro & Char: Analyzing the relationship between pre-training and fine-tuning of large language models. In *Proceedings of the 10th Workshop on Representation Learning for NLP (RepL4NLP)*.

Chupei Wang and Jiaqiu Vince Sun. 2025. Unable to forget: Proactive interference reveals working memory limits in LLMs beyond context length. *arXiv preprint arXiv:2506.08184*.

Kevin Wang, Alexandre Variengien, Arthur Conmy, Buck Shlegeris, and Jacob Steinhardt. 2023. Interpretability in the wild: A circuit for indirect object identification in GPT-2 small. In *International Conference on Learning Representations (ICLR)*.

Jason Wei, Yi Tay, Rishi Bommasani, Colin Raffel, Barret Zoph, Sebastian Borgeaud, Dani Yogatama, Maarten Bosma, Denny Zhou, Donald Metzler, Ed H. Chi, Tatsunori Hashimoto, Oriol Vinyals, Percy Liang, Jeff Dean, and William Fedus. 2022. Emergent abilities of large language models. *Transactions on Machine Learning Research (TMLR)*.

Edwin B. Wilson. 1927. Probable inference, the law of succession, and statistical inference. *Journal of the American Statistical Association*, 22(158):209–212.

Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. 2024. Efficient streaming language models with attention sinks. In *International Conference on Learning Representations (ICLR)*.

Howard Yen, Tianyu Gao, Minmin Hou, Ke Ding, Daniel Fleischer, Peter Izsak, Moshe Wasserblat, and Danqi Chen. 2024. HELMET: How to evaluate long-context language models effectively and thoroughly. *arXiv preprint arXiv:2410.02694*.

Chunting Zhou, Pengfei Liu, Puxin Xu, Srinivasan Iyer, Jiao Sun, Yuning Mao, Xuezhe Ma, Avia Efrat, Ping Yu, Lili Yu, and 1 others. 2023. LIMA: Less is more for alignment. *Advances in Neural Information Processing Systems (NeurIPS)*.

A Dataset Details

We construct two datasets that share the same (K, N) schema and grid but differ in surface form: **Arbitrary-Single (ARB)** uses single-token English words as values, while **Semantic-Multi (SEM)** uses multi-token real category members. Both isolate position-indexed retrieval from semantic shortcuts.

Arbitrary-Single (ARB). A pool of 2,300 single-token English words organised into 46 semantic categories (e.g., *color*, *fruit*, *metal*). For each trial, K categories are sampled as keys and N values are drawn without replacement from each category's pool. Keys and values share no inherent update semantics — there is no reason to expect any particular ordering among values of the same category. This isolates tracking from semantic shortcuts and provides the single-token values required for the mechanistic analysis (§7).

Semantic-Multi (SEM). A pool of 2,403 multi-token values across the same 46 categories, where values are real members of each category (e.g., *gemstone* $\to$ *alexandrite*, *amethyst*, $\ldots$). Values are multi-token and drawn from a smaller per-category pool (45–70 values), limiting feasible update counts to $N \leq 50$ for most key counts. SEM is the primary behavioural dataset because the realistic key–value semantics rule out trivial template-matching strategies.

Grid coverage. On the unified grid $K \in \{2, 3, 5, 7, 10, 15, 20, 25, 30\}$, $N \in \{5, 7, 10, 15, 20, 30, 50, 75, 100\}$ (§3), cells where the prompt exceeds a model's context limit are excluded; cells where the dataset pool is insufficient are also excluded (relevant for SEM at high N). This yields up to 79 feasible cells per model on ARB and up to 63 on SEM.

B Prompts, Matching Rule, and Seeds

This appendix documents the exact text used to construct the inference prompts, the four format variants used in §5, the string-matching rule used to score model responses, and the deterministic seeding scheme.

Naming convention. The main text refers to the second query type as *CVQ* (current-value query), reflecting its meaning. The actual prompt text asks for the *last* value of the target key, which is operationally equivalent when the stream is the complete record of updates.

System prompt and queries

System prompt:
You are a precise data extraction tool. Output ONLY a single word — the exact value requested. No other text, no explanation, no punctuation.

Queries (rendered with the target key substituted in):
FVQ: *What was the first value of* [TARGET]*?*
CVQ: *What was the last value of* [TARGET]*?*
IVQ: *What was the* [K]*-th value of* [TARGET]*?*

Figure 6: Inference-time system prompt and per-query phrasing used in all behavioural sweeps reported in the paper. "CVQ" in the main text corresponds to the literal "last value of" query above.

Stream formats. The four formats compared in §5 differ only in how the stream lines are rendered and which user-turn preamble introduces them. Figure 7 shows a small (K=2, N=3) example in each format.

Chat-template rendering. For instruction-tuned models the user/system messages are rendered through the model's tokenizer-default chat template; for base models without a chat template we use a completion-style few-shot wrapper described in the released code.

Matching rule. A model response is counted as correct if, after lowercasing and stripping whitespace, the expected value either equals the response,

Plain (`flat_nolabel`; default in §4)

Read the following key-value stream. Each key appears multiple times as it gets updated. Count each occurrence of a key as one update.

```
color: red
size: large
color: blue
size: small
color: green
size: medium
```

What was the 3rd value of color?
Answer with ONLY the exact value. No explanation.

Labeled (`flat_verbose`)

Read the following key-value stream. Each key is updated multiple times.

```
color (update 1): red
size (update 1): large
color (update 2): blue
size (update 2): small
color (update 3): green
size (update 3): medium
```

What was the value of color in Update 3?
Answer with ONLY the exact value. No explanation.

Block (`block`)

Read the following key-value stream. Each key is updated multiple times, grouped by update round.

```
[Update 1]
color: red
size: large
[Update 2]
color: blue
size: small
[Update 3]
color: green
size: medium
```

What was the value of color in Update 3?
Answer with ONLY the exact value. No explanation.

Landmark (`landmark`)

Read the following key-value stream. Each key is updated multiple times, grouped by update round. The '—' marker separates consecutive update rounds.

```
color: red
size: large
—
color: blue
size: small
—
color: green
size: medium
```

What was the value of color in Update 3?
Answer with ONLY the exact value. No explanation.

Figure 7: The four prompt formats compared in §5, illustrated on a $K{=}2, N{=}3$ instance. Same underlying stream content; only the surface structure changes. The "Plain" format is the default for all main-body figures and tables (§4).

is contained in the response, or is a prefix of the response. This prefix-tolerant case-insensitive rule accommodates multi-token semantic values where models occasionally emit a trailing qualifier; spot-checks confirm it does not credit lexically unrelated strings.

Seeds. For each $(K, N, \text{query type}, \text{trial index})$ tuple we derive a deterministic seed by hashing the tuple. This ensures the identical stream is presented across base-vs-intervention comparisons at the same cell and trial index, so the two conditions are scored on identical inputs.

C Model Details

Table 7 lists every model evaluated in this paper. The first block is the main open-weight set whose behaviour is analysed in the main text; the second block is three smaller auxiliary models reported in Appendix D for completeness; the third block is the proprietary frontier models. IT = instruction-tuned.

D Full Model Results

Context-usage measurement (Table 1). The **% Ctx** column of Table 1 reports the fraction of each model's advertised context window used by one canonical Semantic-Multi prompt at the row's representative cell. Open-weight rows use the $K{=}5, N{=}30$ prompt ($\mathbf{1{,}029}$ tokens); proprietary rows use the $K{=}20, N{=}50$ prompt ($\mathbf{6{,}565}$ tokens). All counts use `tiktoken o200k_base` (the GPT-4o / GPT-4.1 tokeniser) for uniform cross-provider comparability; on-row tokenisation with each model's actual tokeniser changes individual counts by under $\pm5\%$ for English text and does not move any row above 5% of its window.

Context-window sources. Open-weight windows are read from each model's `config.json` on the HuggingFace Hub (`max_position_embeddings` in `text_config` for multi-modal models): Qwen2.5-3B-Instruct 32,768; Qwen3.5-2B, -4B, -9B 262,144 each; Gemma-3-1b-it 32,768; Gemma-3-4b-it 131,072

Model	Family	Params	IT?	Ctx
Main open-weight				
Qwen2.5-0.5B-Instruct	Qwen2.5	0.5B	✓	32K
Qwen2.5-1.5B-Instruct	Qwen2.5	1.5B	✓	32K
Qwen2.5-3B-Instruct	Qwen2.5	3B	✓	32K
Qwen2.5-3B	Qwen2.5	3B	×	32K
Qwen3.5-0.8B	Qwen3.5	0.8B	✓	256K
Qwen3.5-2B	Qwen3.5	2B	✓	256K
Qwen3.5-4B	Qwen3.5	4B	✓	256K
Qwen3.5-9B	Qwen3.5	9B	✓	256K
Gemma-3-270m-it	Gemma-3	270M	✓	32K
Gemma-3-1b-it	Gemma-3	1B	✓	32K
Gemma-3-4b-it	Gemma-3	4B	✓	128K
Auxiliary (appendix only)				
TinyLlama-1.1B	LLaMA	1.1B	✓	2K
StableLM-2-1.6B	StableLM	1.6B	✓	4K
Pythia-410M	Pythia	410M	×	2K
Proprietary frontier				
GPT-4.1	OpenAI	—	—	1M
GPT-4.1-mini	OpenAI	—	—	1M
Claude-4.5-Haiku	Anthropic	—	—	200K
Claude-4.5-Sonnet	Anthropic	—	—	1M
Gemini-2.5-Flash	Google	—	—	1M
Gemini-2.5-Pro	Google	—	—	1M

Table 7: All 20 models evaluated, grouped by role: 11 main open-weight models that drive the main analyses; 3 smaller auxiliary models reported only in Appendix D; and 6 proprietary frontier models. **Ctx** is the advertised input context window (HF `max_position_embeddings` for open-weight; provider documentation for proprietary; see Appendix D).

(rope-scaling factor 8 over the 16,384-token base). Proprietary windows are advertised input limits from provider documentation: GPT-4.1 and GPT-4.1-mini 1,000,000 tokens (`https://openai.com/index/gpt-4-1/`); Claude-4.5-Haiku 200,000 tokens; Claude-4.5-Sonnet 1,000,000 tokens via the long-context option (`https://docs.anthropic.com/en/docs/about-claude/models/overview`); Gemini-2.5-Flash 1,048,576 and Gemini-2.5-Pro 1,000,000 (`https://ai.google.dev/gemini-api/docs/models`).

⟨**NOTE:** Carry over: per-model SEM grid + proprietary table.⟩

E Arbitrary-Single Replication

⟨**NOTE:** Carry over: ARB grid table.⟩

F Format Intervention Data

This appendix supports two claims made in the main text: the format-intervention results in §5.1 (Table 2, in §5.1) and the multi-model intermediate-position retrieval claim in §4.

Multi-model per-position retrieval under the default Plain format. Figure 8 shows per-position retrieval accuracy at $K{=}10, N{=}50$ on Arbitrary-Single under the default Plain format, separately for the five open-weight models in the ucurve sweep and the six proprietary frontier models. All five open-weight models collapse below 5% accuracy within two positions of the stream start. The proprietary panel reveals three regimes: three models (GPT-4.1, GPT-4.1-mini, Claude-4.5-Haiku) collapse like the open-weight set; Claude-4.5-Sonnet and Gemini-2.5-Flash show partial recovery; Gemini-2.5-Pro alone maintains near-ceiling accuracy across all queried positions.

G Training Dynamics Data

This appendix lists the per-checkpoint mean gap (and component accuracies) that underlie Figure 4 in §6.

Method. Each checkpoint is evaluated on Arbitrary-Single with the same query format and matching rule used in the main paper (Appendix B), at $K \in \{2,3,5,7\}$, $N \in \{2,3,5,7,10,15,20,30\}$, 100 trials per cell. The completion-format prompt is used at every checkpoint to ensure comparability across pretraining and post-training stages; the chat-template prompt returns near-zero accuracy on early instruction-tuned SmolLM3 checkpoints (`it-mid-training` under `chat_1024` gives mean RI = mean PI = 0.001), so we report it only as a robustness comparison (Table 10). For figures that partition by load, the 32 cells split into three exhaustive bins: low ($K{\leq}3$, any N; 16 cells), medium ($K{\geq}5, N{<}20$; 12 cells), and high ($K{\geq}5, N{\geq}20$; 4 cells).

SmolLM2-1.7B trajectory. Figure 9 reports the SmolLM2-1.7B trajectory under the same three-bin partition used for SmolLM3 in §6. The pattern matches SmolLM3 with a cleaner separation: low-load cells stay positive across the published four-stage curriculum (Ben Allal et al., 2025), medium-load cells hover near zero, and high-load cells stay in the reversal direction throughout pretraining and into the released final model. Table 8 reports the per-checkpoint mean RI (FVQ), PI (CVQ), and gap at every released checkpoint. Aggregated across all 32 cells the gap is positive at every checkpoint, with a non-monotonic curriculum trajectory — rising through early pretraining, peaking around step 625k, partially collapsing, then stabilising near the

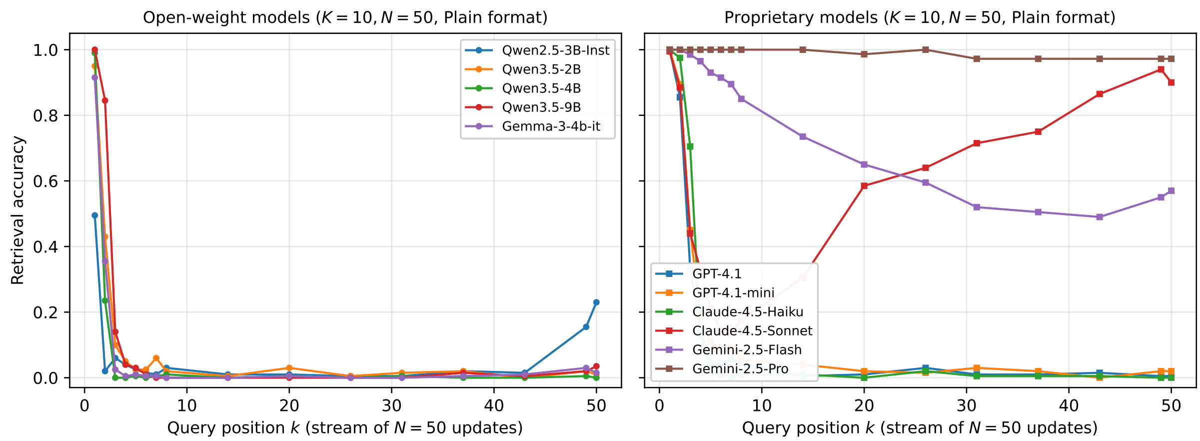

Figure 8: Per-position retrieval accuracy on Arbitrary-Single at $K{=}10, N{=}50$ under the default Plain format (ucurve sweep; `flat_nolabel` condition). Each line is one model. **Left**: five open-weight models, all collapsing uniformly to near-floor by position 2. **Right**: six proprietary models; most collapse but Gemini-2.5-Pro maintains near-ceiling accuracy throughout, and Claude-4.5-Sonnet shows U-shaped recovery toward the end of the stream.

final value.

SmolLM3-3B trajectory. Table 9 reports the same statistics for SmolLM3-3B across 32 pretraining checkpoints (the three documented stages) and the four post-training stages plus the released final model. The gap is positive at every checkpoint and at every post-training stage including `final`.

Prompt-format robustness check for post-training. For the four SmolLM3 post-training stages plus the released model we additionally ran the chat-template prompt (Table 10). Magnitudes differ between prompt formats but the sign and ordering of the gap are preserved at every stage where the chat template itself does not break the model (at `it-mid-training` the chat template gives near-zero accuracy on both queries, so the gap is undefined and we omit it from Figure 4).

H LoRA Training Details

This appendix documents the full configuration of the LoRA intervention reported in §5.2. All scripts, training data, and checkpoint selection logic are in the released codebase.

Adapter and optimiser. The adapter is fitted with PEFT (Hu et al., 2022) on top of the frozen base model. LoRA matrices are inserted into the four attention projections $\{Q, K, V, O\}$ of every transformer layer; MLP and embedding weights are left unmodified. Hyperparameters are listed in Table 11.

Training data composition. Each training example is one (K, N) stream with a single query appended. The 18,000 training examples are split 40 % FVQ, 40 % CVQ, 20 % intermediate-position queries (IVQ). Including IVQ during training removes a possible shortcut: a model that simply amplified a recency-attending circuit could pass FVQ plus CVQ supervision while leaving intermediate positions unchanged; IVQ supervision blocks that path. Training examples are sampled uniformly across the 16 training cells, yielding ~1,125 examples per cell.

Training and held-out cell split. The training and evaluation cells are disjoint by design. Table 12 lists both grids; held-out cells extend roughly 10× beyond training in stream length ($K \times N$ from ≤ 200 at training to $\leq 2{,}250$ at evaluation).

Category split. Of the 46 semantic categories in the Arbitrary-Single pool (Appendix A), 35 are used during training and 11 are held out for evaluation; no category appears in both splits (Table 13).

Compute. End-to-end training (18k examples, two epochs, effective batch 64) runs to completion on a single L40S GPU in approximately 1.5 hours, including evaluation passes after each epoch.

Pre-registered controls (⟨**NOTE: both pending**⟩)**.** We pre-registered two controls before running the main experiment. A **family control** trains the identical adapter configuration (rank, learning rate, schedule, data mix) on Gemma-3-4b-it; success here indicates the recovery is architecture-agnostic

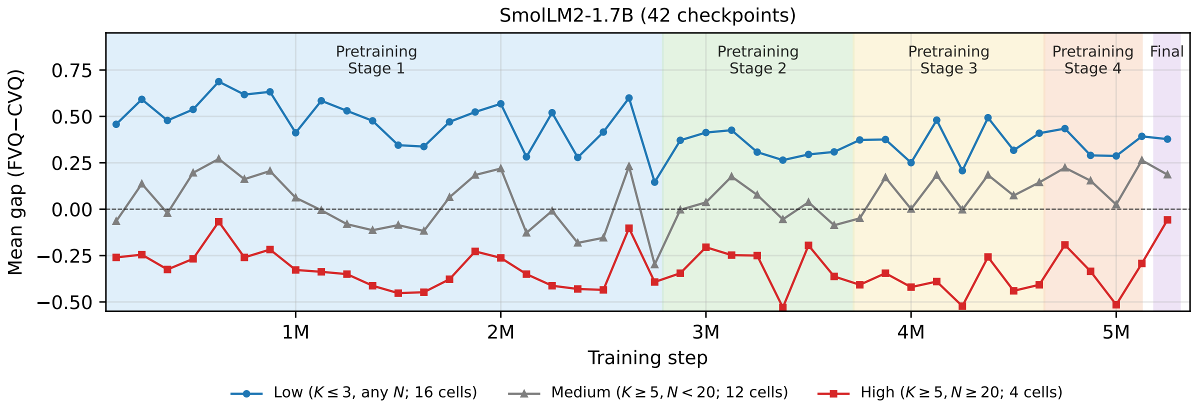

Figure 9: Mean FVQ−CVQ gap on Arbitrary-Single across the SmolLM2-1.7B (Ben Allal et al., 2025) training trajectory, under the same three-bin partition as Figure 4. Coloured bands mark the four documented pretraining stages plus the released final model.

rather than specific to Qwen2.5. A **negative control** fine-tunes the same Qwen2.5-3B-Instruct with identical hyperparameters on an arithmetic-word-problem dataset of comparable size, then evaluates on the FVQ/CVQ grid; if the gap closes here, our test setup is contaminated by something general about LoRA training, and the claim that the recovery is task-specific is undermined. Both controls are pending; we will update the main text with the result when complete.

Full held-out results. Table 14 reports the full FVQ and CVQ accuracies that underlie the compact gap-only Table 3 in §5.2.

I Dense Intermediate-Position Retrieval — Full Sweep

Figure 3 in §5 shows position-by-position retrieval accuracy at a single cell (K=5, N=10). To substantiate the broader claim that the failure is a general loss of position-indexed retrieval, we report the same measurement across all six cells we tested. Figure 10 shows the per-position curves; Table 15 summarises first / intermediate / last position accuracy per cell. Across all six (K, N) cells, baseline intermediate-position accuracy stays at or below 25% and the LoRA-tuned model recovers near-ceiling accuracy at every position queried.

J Pre/Post Mechanism Details

⟨**NOTE:** New: pre/post probing, logit-lens, attention-routing figures and tables (pending experiments).⟩

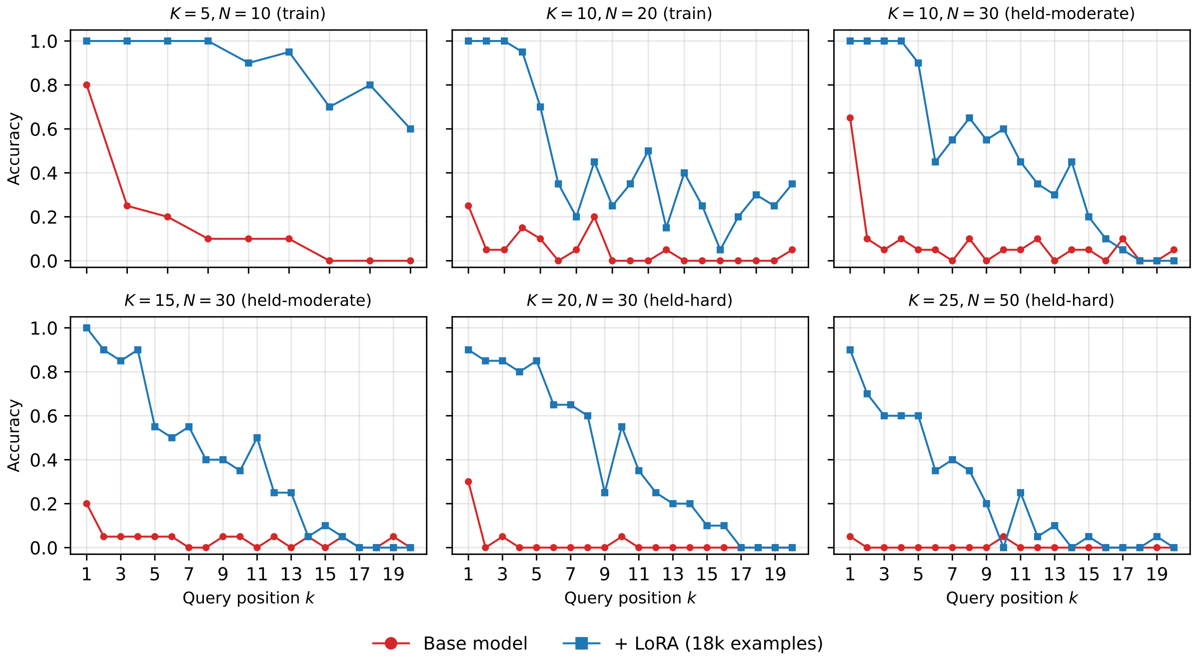

Figure 10: Per-position retrieval accuracy at all six tested (K, N) cells on Arbitrary-Single, base model vs LoRA-tuned model, for positions $k \in [1, N-1]$ (k=N is omitted because the experiment script substitutes a CVQ "last value" query at that position). Group labels (`train`, `held-moderate`, `held-hard`) indicate whether the cell was included in the LoRA training grid.

Step	RI (FVQ)	PI (CVQ)	Gap	Cells
125,000	0.37	0.20	+0.17	32
250,000	0.50	0.18	+0.32	32
375,000	0.49	0.30	+0.19	32
500,000	0.56	0.25	+0.31	32
625,000	0.58	0.14	+0.44	32
750,000	0.58	0.24	+0.34	32
875,000	0.59	0.22	+0.37	32
1,000,000	0.55	0.37	+0.19	32
1,125,000	0.57	0.32	+0.25	32
1,250,000	0.52	0.33	+0.19	32
1,375,000	0.50	0.36	+0.14	32
1,500,000	0.53	0.45	+0.08	32
1,625,000	0.48	0.41	+0.07	32
1,750,000	0.52	0.31	+0.21	32
1,875,000	0.56	0.26	+0.30	32
2,000,000	0.57	0.23	+0.33	32
2,125,000	0.40	0.35	+0.05	32
2,250,000	0.55	0.34	+0.20	32
2,375,000	0.46	0.44	+0.02	32
2,500,000	0.52	0.43	+0.10	32
2,625,000	0.57	0.19	+0.37	32
2,750,000	0.38	0.47	−0.09	32
2,875,000	0.51	0.37	+0.14	32
3,000,000	0.53	0.33	+0.19	32
3,125,000	0.61	0.36	+0.25	32
3,250,000	0.59	0.44	+0.15	32
3,375,000	0.55	0.51	+0.05	32
3,500,000	0.58	0.44	+0.14	32
3,625,000	0.56	0.48	+0.08	32
3,750,000	0.58	0.46	+0.12	32
3,875,000	0.66	0.45	+0.21	32
4,000,000	0.58	0.51	+0.07	32
4,125,000	0.65	0.39	+0.26	32
4,250,000	0.63	0.59	+0.04	32
4,375,000	0.62	0.34	+0.28	32
4,500,000	0.60	0.47	+0.13	32
4,625,000	0.62	0.41	+0.21	32
4,750,000	0.70	0.43	+0.28	32
4,875,000	0.67	0.50	+0.16	32
5,000,000	0.64	0.55	+0.09	32
5,125,000	0.73	0.47	+0.26	32
final	0.75	0.50	+0.25	32

Table 8: SmolLM2-1.7B per-checkpoint mean RI (FVQ), PI (CVQ), and gap on Arbitrary-Single. 32 cells per checkpoint; 100 trials/cell.

Checkpoint	RI (FVQ)	PI (CVQ)	Gap	Cells
Pretraining (completion_few_shot)				
stage1-step-40,000	0.66	0.07	+0.59	32
stage1-step-160,000	0.74	0.40	+0.34	32
stage1-step-320,000	0.86	0.33	+0.53	32
stage1-step-440,000	0.88	0.43	+0.45	32
stage1-step-600,000	0.81	0.42	+0.40	32
stage1-step-1,160,000	0.76	0.33	+0.42	32
stage1-step-1,280,000	0.72	0.45	+0.27	32
stage1-step-1,440,000	0.83	0.43	+0.40	32
stage1-step-1,560,000	0.79	0.45	+0.33	32
stage1-step-1,720,000	0.72	0.33	+0.39	32
stage1-step-1,880,000	0.62	0.35	+0.27	32
stage1-step-2,000,000	0.69	0.51	+0.18	32
stage1-step-2,160,000	0.61	0.55	+0.07	32
stage1-step-2,280,000	0.72	0.40	+0.32	32
stage1-step-2,440,000	0.66	0.60	+0.06	32
stage1-step-2,560,000	0.76	0.46	+0.30	32
stage1-step-2,720,000	0.79	0.31	+0.48	32
stage1-step-2,840,000	0.85	0.30	+0.55	32
stage1-step-3,000,000	0.79	0.51	+0.28	32
stage1-step-3,120,000	0.77	0.59	+0.17	32
stage1-step-3,280,000	0.68	0.59	+0.09	32
stage1-step-3,440,000	0.70	0.57	+0.13	32
stage2-step-3,480,000	0.75	0.43	+0.32	32
stage2-step-3,640,000	0.75	0.53	+0.22	32
stage2-step-3,840,000	0.75	0.57	+0.18	32
stage2-step-4,000,000	0.82	0.71	+0.11	32
stage2-step-4,200,000	0.80	0.55	+0.25	32
stage3-step-4,240,000	0.83	0.48	+0.35	32
stage3-step-4,360,000	0.82	0.49	+0.32	32
stage3-step-4,480,000	0.80	0.44	+0.36	32
stage3-step-4,600,000	0.82	0.49	+0.33	32
stage3-step-4,720,000	0.80	0.58	+0.22	32
Post-training (completion)				
it-mid-training	0.79	0.53	+0.25	32
it-SFT	0.82	0.63	+0.19	32
it-soup-APO	0.80	0.61	+0.19	32
it-LC-expert	0.83	0.55	+0.27	32
final	0.81	0.64	+0.16	32

Table 9: SmolLM3-3B per-checkpoint mean RI (FVQ), PI (CVQ), and gap on Arbitrary-Single (completion format throughout). 32 cells per checkpoint; 100 trials/cell.

	Completion		Chat (1024)	
Stage	Gap	Cells	Gap	Cells
it-mid-training	+0.25	32	−0.00	32
it-SFT	+0.19	32	+0.29	32
it-soup-APO	+0.19	32	+0.20	32
it-LC-expert	+0.27	32	+0.33	32
final	+0.16	32	+0.18	32

Table 10: SmolLM3-3B post-training stages: mean gap under `completion` vs `chat_1024` prompt formats. The chat template returns near-zero accuracy on the `it-mid-training` checkpoint; magnitudes shift at later stages but the gap stays positive under both formats.

Setting	**Value**
Base model	Qwen2.5-3B-Instruct
LoRA rank	16
LoRA α	32
LoRA dropout	0.05
Target modules	q_proj, k_proj, v_proj, o_proj
Trainable parameters	7.4M (0.24% of model)
Learning rate	2×10^{-4}
LR schedule	cosine decay, 100-step linear warm-up
Optimiser	paged AdamW (8-bit)
Epochs	2
Batch size	8 (×8 grad accum = effective 64)
Max sequence length	2,048
Mixed precision	bfloat16
Training examples	18,000 (+2,000 validation, same cells, different seeds)
Checkpoint selected	best validation loss (step 564, end of epoch 2)

Table 11: LoRA training hyperparameters.

Split	**Cells** (K, N)
Training (16 cells)	$K \in \{2, 3, 5, 10\}$, $N \in \{5, 10, 15, 20\}$
Held-out (28 cells)	K=7, $N \in \{30, 50, 75\}$
	K=10, $N \in \{30, 50, 75\}$
	K=15, $N \in \{15, 20, 30, 50, 75\}$
	K=20, $N \in \{15, 20, 30, 50, 75\}$
	K=25, $N \in \{10, 15, 20, 30, 50, 75\}$
	K=30, $N \in \{10, 15, 20, 30, 50, 75\}$

Table 12: Training and held-out cells for the LoRA intervention. Training cells cover only low (K, N); held-out cells extend to K=30 and N=75, well outside the training distribution in both dimensions.

Training (35)	**Held-out (11)**
visual art, tools, landform, musical instrument, gemstone, fabric, tree species, cheese variety, architectural style, cloud formation, bird species, culinary herb, flower species, wine variety, dance style, pasta shape, literary genre, cooking method, mathematical concept, weather phenomenon, ocean current, mineral type, coffee variety, telescope type, martial art, sea creature, psychology term, chemical element, dinosaur genus, programming language, ancient civilization, bridge type, photography technique, boat type, cartoon character	*stadium name, surgical procedure, constellation, spice blend, guitar type, hat style, painting medium, volcano name, fruit variety, sword type, board game*

Table 13: LoRA category split. 35 categories used during training, 11 held out for evaluation; no overlap.

	Base			**+ LoRA**		
(K, N)	FVQ	CVQ	gap	FVQ	CVQ	gap
$(7, 30)$	50%	38%	+12%	100%	100%	+0%
$(7, 50)$	52%	46%	+6%	100%	96%	+4%
$(10, 30)$	41%	46%	−5%	100%	100%	+0%
$(10, 50)$	42%	50%	−8%	100%	100%	+0%
$(15, 20)$	32%	58%	−26%	100%	98%	+2%
$(15, 30)$	35%	60%	−25%	100%	98%	+2%
$(15, 50)$	18%	59%	−41%	100%	100%	+0%
$(20, 30)$	21%	51%	−30%	100%	98%	+2%
$(20, 50)$	8%	62%	−54%	100%	96%	+4%
$(20, 75)$	6%	56%	−50%	100%	96%	+4%
$(25, 30)$	15%	59%	−44%	100%	98%	+2%
$(25, 50)$	5%	43%	−38%	98%	96%	+2%
$(25, 75)$	6%	60%	−54%	100%	93%	+7%
$(30, 30)$	12%	60%	−48%	100%	98%	+2%
$(30, 50)$	4%	54%	−50%	100%	94%	+6%
$(30, 75)$	1%	56%	−55%	99%	92%	+6%

Table 14: Full held-out Arbitrary-Single results for the LoRA on Qwen2.5-3B-Instruct. Same 16 cells as Table 3; this appendix table reports FVQ and CVQ accuracy separately alongside the gap, before and after the adapter.

	Base			**+ LoRA**		
(K, N)	Pos. 1	Interm. min–max	Last	Pos. 1	Interm. min–max	Last
$(5, 10)$	80%	0%–25%	0%	100%	70%–100%	60%
$(10, 20)$	25%	0%–20%	5%	100%	5%–100%	35%
$(10, 30)$	65%	0%–10%	5%	100%	0%–100%	0%
$(15, 30)$	20%	0%–5%	0%	100%	0%–90%	0%
$(20, 30)$	30%	0%–5%	0%	90%	0%–85%	0%
$(25, 50)$	5%	0%–5%	0%	90%	0%–70%	0%

Table 15: Per-cell summary of dense intermediate-position retrieval accuracy on Arbitrary-Single. “Interm. min–max” is the range across all intermediate positions ($1 < k < N$) at that cell. The base model loses accuracy at every intermediate position across every cell; the LoRA-tuned model holds near ceiling.